Web-Controlled Rescue Robotic Vehicle: Design, Implementation, and Environmental Monitoring System

Nguyen Tran Quang (SE200988)[1], Nguyen Tan Tai (SE201664)[2],
and Le Nguyen Nhat Huy (SE200020)[3]
Team 4, Class IC2010, Microcontroller Programming
FPT University
Ho Chi Minh City, Vietnam
Lecturer: Sir. Pham The Vinh

***Abstract*—This project presents the design, construction, and programming of a cost-effective, Wi-Fi-controlled rescue robotic vehicle using the ESP32 microcontroller. Designed to assist in hazardous environments typical of Small and Medium-sized Enterprises (SMEs), the robot provides real-time monitoring to prevent human exposure to toxic gas leaks. Unlike standard Bluetooth-controlled vehicles, this system features a custom local Web Server (SoftAP) for platform-independent control. It utilizes Asynchronous JavaScript and XML (AJAX) to stream live telemetry from an MQ-2 gas sensor and HC-SR04 ultrasonic sensor. To ensure smooth multitasking without Wi-Fi latency or microcontroller freezing, the system employs a Non-Blocking Finite State Machine (FSM) architecture. Experimental results validate the system's low latency, power stability, and reliable obstacle avoidance. The report also outlines clear hardware and software roadmaps for resolving current mechanical drift and achieving fully autonomous navigation.**

***Index Terms*—ESP32, Unmanned Ground Vehicle (UGV), Finite State Machine (FSM), AJAX Telemetry, Gas Detection, Motor Control, Web Server.**

I. INTRODUCTION

Fig. 1. Conceptual overview of the Rescue Robot Car. *(Source: Image by Aslam Hossain)*

A. Background

During our research on industrial safety, we found that many small factories (SMEs) still lack automatic gas monitoring systems, leading to high risks of accidents like toxic gas leaks (e.g., LPG, CH4, CO) and fires. In these hazardous situations, sending rescue teams directly into unknown areas without prior data is dangerous and can lead to secondary casualties.

Unmanned Ground Vehicles (UGVs) offer a practical solution. By entering first, these robots can gather real-time air quality data and locate leak sources. However, commercial inspection robots are often too expensive and complex for most SMEs. This project addresses this gap by developing a budget-friendly, Wi-Fi-controlled robotic vehicle designed to meet the basic safety and reconnaissance needs of smaller enterprises.

B. Key Improvements

Compared to typical student projects that rely on basic Bluetooth modules, this project features three main technical improvements:

- Wi-Fi Web Control: Instead of requiring a custom mobile application, the ESP32 creates its own Wi-Fi Access Point (SoftAP). Users can control the vehicle instantly using a standard web browser on any smartphone or laptop.
- Live Sensor Data: The vehicle uses an MQ-2 gas sensor and an ultrasonic sensor to monitor its surroundings. The web interface uses AJAX to continuously update gas levels and distance metrics in real-time without refreshing the page.
- Smooth Multitasking: To maintain a stable Wi-Fi connection while driving, we refactored the code using a Non-Blocking Finite State Machine (FSM). This allows the robot to drive, process sensor data, and maintain the web server concurrently without crashing.

C. Research Objectives

- Develop a Universal Control Interface: To build a system using the ESP32 as a standalone Web Server, allowing control from any device without third-party apps.
- Enable Real-Time Environmental Monitoring: To integrate an MQ-2 and an ultrasonic sensor to measure toxic gas and detect obstacles, updated live via AJAX.

- Implement Non-Blocking Multitasking: To develop a stable software architecture (FSM) that allows the simultaneous execution of motor control and sensor polling.
- Provide an Obstacle Avoidance Mode: To program a self-driving feature where the robot automatically detects walls, stops, reverses, and changes direction.

II. THEORETICAL AND PRACTICAL BACKGROUND

A. Core Concepts

Our rescue robot is a compact UGV built to enter tight or dangerous spaces. It can gather environmental data and perform basic tasks through remote control or autonomous programming. Like most robots, our vehicle operates on a simple cycle: reading sensors (Sense), calculating the next move (Process), and driving the motors (Act).

To achieve this, the vehicle is constructed from five basic modules: Control Module, Actuation Module, Sensor Module, Communication Module, and Power Supply Module.

B. Operating Principles

When powered on, the ESP32 creates its local Wi-Fi network (SoftAP). When the user presses a direction button on the webpage, the browser sends an HTTP request to the ESP32. The ESP32 processes this and sends PWM signals to the L298N motor driver to turn the wheels. Simultaneously, the ESP32 reads analog voltage from the MQ-2 and digital pulses from the ultrasonic sensor. The webpage uses AJAX to request this data every 500 milliseconds, instantly updating the user's screen.

C. Overview of Implementation Steps

- Hardware Assembly & Wiring: Mounting the 4 DC motors and cross-wiring them to the L298N. Connecting the ESP32 and sensors to the power circuit.
- Software Development: Programming the ESP32 to set up the Wi-Fi, read sensors, and control motors using a non-blocking state machine (`millis()`).
- Web Interface Design: Creating the HTML/CSS D-Pad and writing JavaScript for AJAX requests.
- Testing and Calibration: Testing Wi-Fi range, adjusting the MQ-2 threshold, and adjusting motor timing for turns.

III. HARDWARE DESIGN AND IMPLEMENTATION

A. List of Components

- Main Microcontroller: ESP32 Development Board.
- Motor Driver: L298N Dual H-Bridge.
- Actuators & Chassis: 4WD Robot chassis kit with 4x DC Gear Motors (Yellow TT Motors) and SG90 Micro Servo.
- Sensors: MQ-2 Analog Gas Sensor and HC-SR04 Ultrasonic Sensor.
- Power Supply: 2x 18650 Lithium-ion batteries.

B. Block Diagram

As shown in the block diagram (Figure 2), the ESP32 acts as the main controller. It continuously collects data from the MQ-2 and ultrasonic sensors. Based on these inputs and user commands, it outputs PWM signals to the L298N to operate the motors. By using this architecture, the ESP32 can read sensors, update the webpage, and drive the motors at the same time without memory overflows.

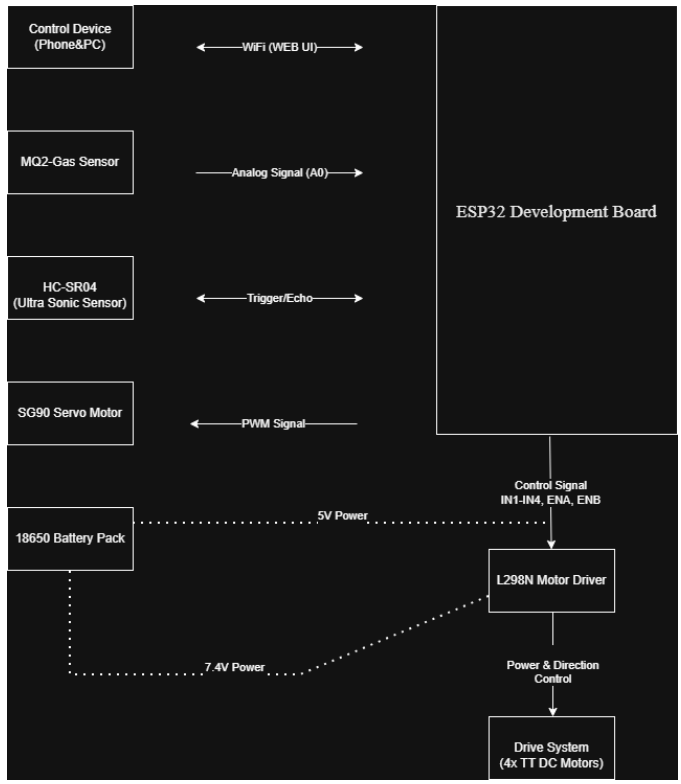

Fig. 2. System block diagram illustrating hardware connections.

C. Circuit Diagram and Wiring Connection

The hardware was wired as follows (Figure 3):

Power Supply Distribution:

- 18650 Battery Pack (+) → L298N (12V Input)
- 18650 Battery Pack (-) → L298N (GND)
- L298N (5V Output) → ESP32 (VIN / 5V)
- L298N (GND) → ESP32 (GND)

Drive System (DC Motors to L298N):

- Left Motors (Red/Black) → L298N (OUT1 / OUT2)
- Right Motors (Red/Black) → L298N (OUT4 / OUT3)

Motor Control Signals (L298N to ESP32):

- L298N (ENA) → ESP32 (GPIO 14) [Left PWM]
- L298N (IN1 / IN2) → ESP32 (GPIO 27 / 26) [Left Dir]
- L298N (IN3 / IN4) → ESP32 (GPIO 25 / 33) [Right Dir]
- L298N (ENB) → ESP32 (GPIO 32) [Right PWM]

HC-SR04 Sensor: TRIG → GPIO 4; ECHO → GPIO 5.

MQ-2 Gas Sensor: A0 → GPIO 36 (VP). Powered by L298N 5V.

SG90 Servo: Signal → GPIO 13. Powered by L298N 5V.

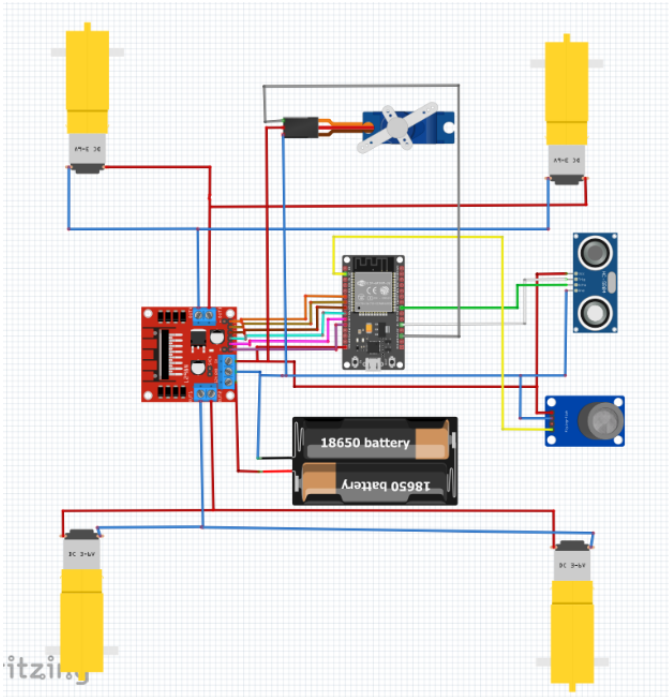

Fig. 3. Detailed circuit schematic and component wiring.

IV. SOFTWARE PROGRAMMING AND USER DASHBOARD

A. Software Architecture

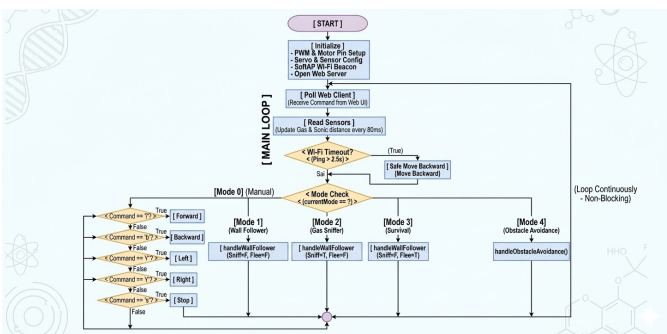

Fig. 4. Software architecture and Finite State Machine flowchart.

The core software relies on a Non-Blocking Finite State Machine (Figure 4). During the initial testing phase, we found that using `delay()` caused the web dashboard to disconnect frequently. To maintain a stable Wi-Fi connection, we refactored the code using `millis()` for non-blocking execution. This allows the ESP32 to poll sensors every 80ms and execute motor logic without freezing.

B. Feature Algorithms and Development Status

The software logic is divided into five modes based on HTTP GET requests:

Fully Implemented Features:

- Mode 0 (Manual): Provides zero-latency manual control over the drivetrain and the servo via the web dashboard.
- Mode 4 (Obstacle Avoidance): Upon detecting a frontal obstacle (< 30cm), the robot halts and turns the SG90 servo to sweep right (10°) and left (170°). It compares the left and right distances, steers toward the wider path, and makes a 180-degree U-turn if both paths are dead ends.

Features Under Active Development:

- Mode 1 (Wall Follower): Designed to map environments using a move-and-stop approach. Currently, wheel slip on smooth surfaces causes inaccuracies, requiring future calibration.
- Mode 2 (Gas Seeker) & Mode 3 (Survivor): Mode 2 tracks leaks by comparing left/right gas levels. Mode 3 executes a 180-degree evasion at critical gas thresholds. Future work will incorporate digital low-pass filters (Kalman) and rotary encoders to improve movement accuracy.

C. Web Server Interface and User Dashboard

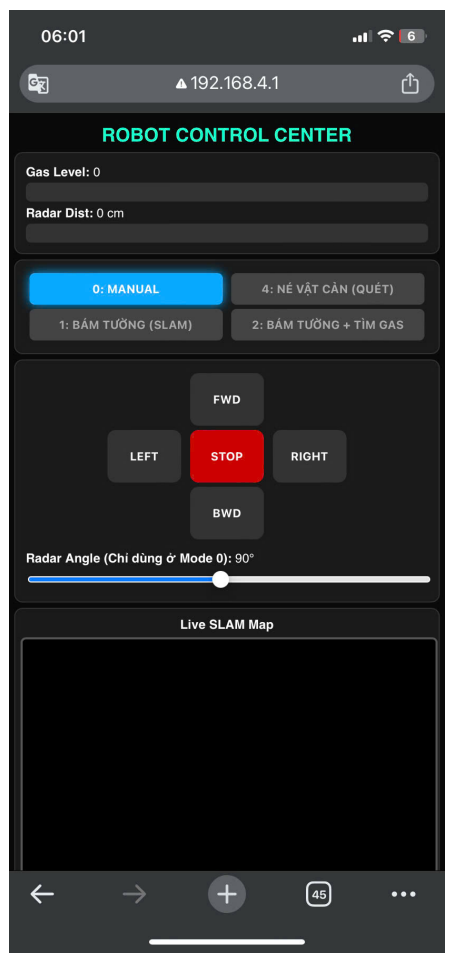

Fig. 5. User interface of the custom web-based control dashboard.

The ESP32 is configured as a standalone SoftAP. By storing the HTML/CSS in the ESP32's PROGMEM, we made the system platform-independent (Figure 5).

- AJAX Telemetry: The client-side dashboard uses the JavaScript Fetch API for asynchronous polling every 500ms, displaying live gas and distance levels without refreshing.
- Client-Side Processing (2D Mapping): To conserve the ESP32's memory, the math for 2D mapping is offloaded to the client's device. The JavaScript engine plots the estimated trajectory on an HTML5 `<canvas>`. While functional, current map drawing is sometimes inaccurate due to motor sliding.

D. Source Code

Available at: https://github.com/The-Beacrox/Rescue-Robot-Car/blob/main/RescueRoBotCarCode.ino

V. SYSTEM OPERATION

Step 1: Hardware Initialization

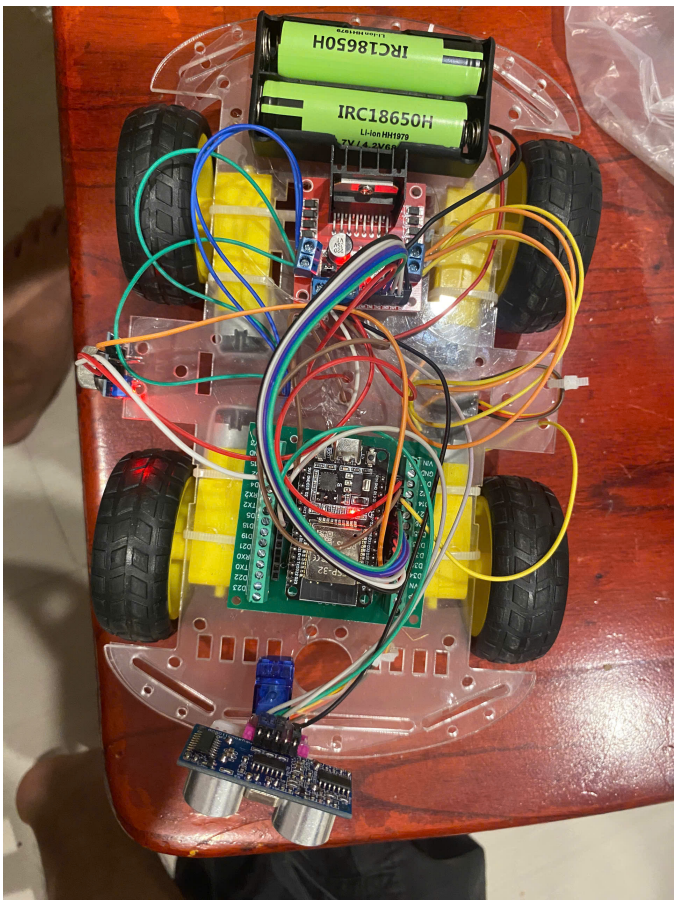

Fig. 6. Hardware initialization state upon power-up.

Ensure the 18650 battery pack is charged. Toggle the power switch to ON. Wait 3 to 5 seconds. The ESP32 will initialize sensors, center the servo, and broadcast its SoftAP (Figure 6).

Step 2: Network Connection On a smartphone/laptop, scan for Wi-Fi networks and connect to the SSID: `Robot_Tham_Do`. Password: `thebeacroxtheic161093` (Figure 7).

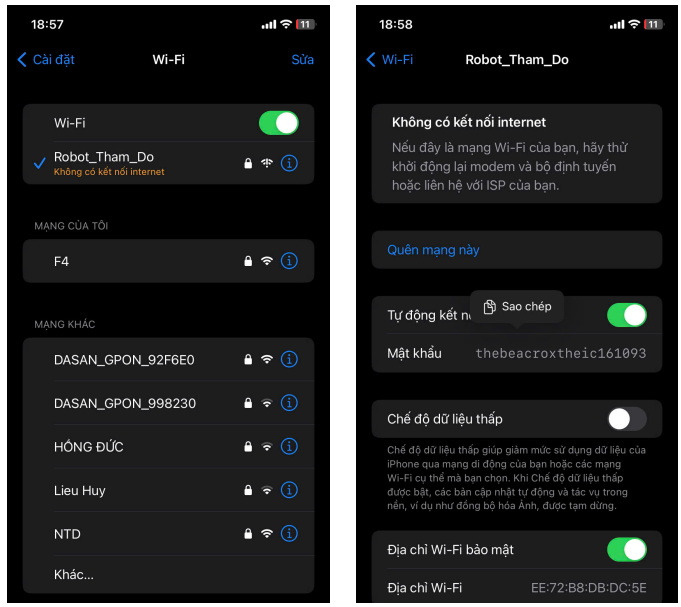

Fig. 7. Connecting to the ESP32 SoftAP Wi-Fi network.

Step 3: Accessing the Web Dashboard

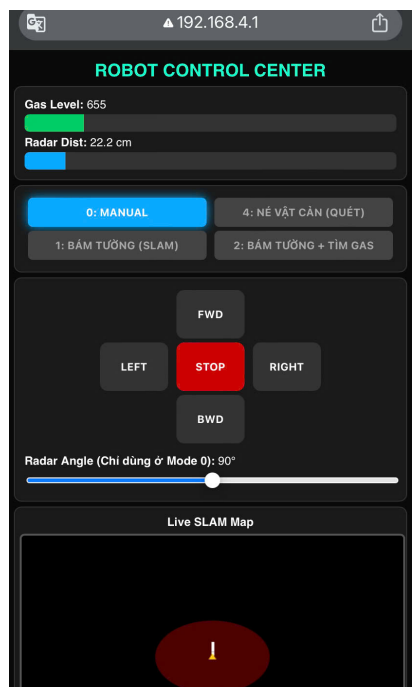

Fig. 8. Real-time telemetry display on the Control Center dashboard.

Open a standard browser and type the IP address http://192.168.4.1. The "ROBOT CONTROL CENTER" will load immediately, updating telemetry every 500ms (Figure 8).

Step 4: Operating the Robot

- Telemetry Panel: Monitor color-coded bars for "Gas Level" and "Radar Dist".

- Mode Selection: Click to switch states: `0: MANUAL`, `4: Obstacle Avoidance`, `1: Wall Follower`, `2: Gas Seeker`, `3: Survivor`.
- Manual Controls: Use D-Pad buttons to drive. Use the Slider to rotate the radar.
- 2D Map: Visually track the estimated path (Blue line) and hazard zones.

VI. EXPERIMENTAL RESULTS AND FUTURE WORK

A. Experimental Results

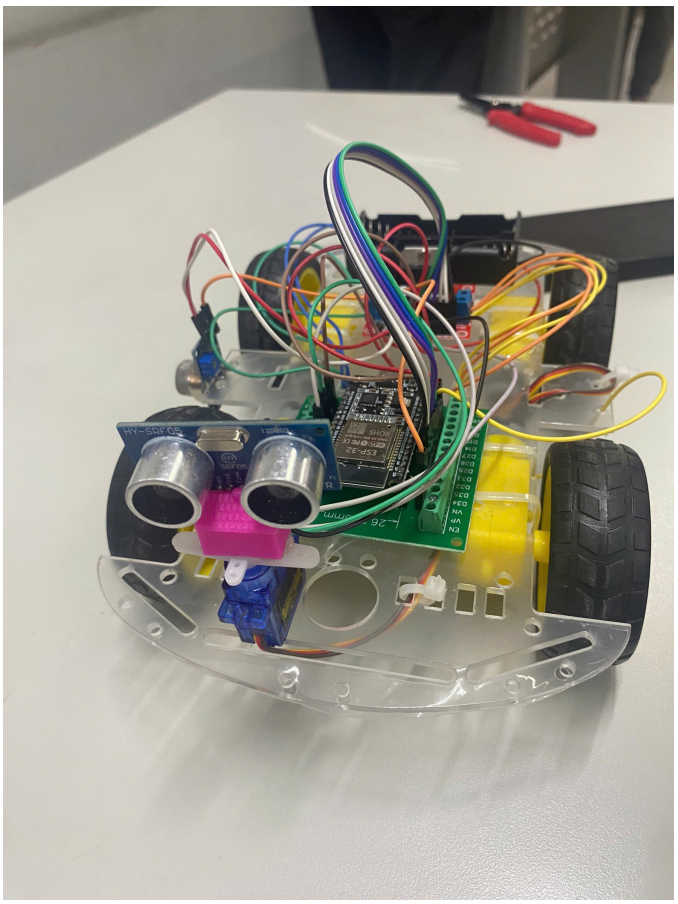

Fig. 9. The fully assembled rescue robot prototype during experimental testing.

The robot was tested in a classroom environment (Figure 9).

- Connectivity: The SoftAP maintained a stable Wi-Fi connection up to 15 meters without significant packet loss. AJAX telemetry updated with an average latency of 100ms - 150ms.
- Obstacle Avoidance: During 10 trial runs at 50% PWM speed, the HC-SR04 successfully detected walls at the 30cm threshold and successfully avoided the wall 9 out of 10 times.
- Gas Calibration: Through manual calibration, the warning threshold was set to a reliable analog value that filtered out background noise, accurately triggering alerts when exposed to a direct gas source (lighter).
- System Stability: The FSM architecture prevented system crashes during a continuous 30-minute operational test.

B. Future Work

Future iterations will focus on:

- Refining Navigation: Perfecting the move-and-stop approach to reduce ultrasonic errors for better HTML5 canvas mapping.
- Chemical Tracking: Stabilizing Mode 2 and Mode 3 logic to ensure the robot reliably pivots and escapes when gas exceeds the 1800-unit critical threshold.
- Hardware Enhancements: Implementing rotary encoders on the wheels to replace time-based movement, significantly reducing tracking drift.
- Vision Integration & SLAM: Reintegrating an ESP32-CAM module for live video (OpenCV) and upgrading to grid-based occupancy mapping (A^* pathfinding).

VII. CONCLUSION

This project successfully designed and implemented a universal, web-controlled rescue robot using the ESP32. By leveraging a non-blocking FSM and AJAX, the system achieves stable real-time monitoring and reliable basic autonomous navigation without Wi-Fi latency. While advanced features require further hardware stabilization (e.g., encoders to fix wheel slip), the current prototype establishes a solid working baseline. The integration of continuous sensor polling and a platform-independent interface shows that a low-cost ESP32 system can be highly useful for basic safety inspections in small factories.

REFERENCES

[1] R. Santos and S. Santos, Learn ESP32 with Arduino IDE: Practical Projects to Build your own IoT Systems. Random Nerd Tutorials, 2019.

[2] M. Schwartz, Internet of Things with ESP32: Build Web-based smart robots and IoT applications. Birmingham, UK: Packt Publishing, 2016.

[3] YouTube. (2023). ESP32 Web Controlled Robot Car with Obstacle Avoidance. https://youtu.be/toWPdFhwAHM

[4] YouTube. (2023). ESP32 Robot Car Complete Tutorial. https://youtu.be/ZjxEfAh5XB8

[5] YouTube. (2024). Autonomous Obstacle Avoiding Robot using Ultrasonic Sensor. https://youtu.be/YqUr6wVHAyo